

Guía: Collections en Java — ArrayList, HashSet y HashMap

Tabla comparativa rápida

Estructura	Interfaz	Permite duplicados	Orden garantizado	Acceso por índice/clave	Complejidad promedio (get/add)
ArrayList	List	Sí	Sí (inserción)	Índice (int)	O(1) amortizado add; O(1) get
HashSet	Set	No	No (no garantizado)	No (búsqueda por objeto)	O(1) add/contains
HashMap	Map	Clave única; valores duplicados	No (no garantizado)	Clave → Valor	O(1) put/get

Diferencias clave (texto)

- **ArrayList**: lista dinámica basada en array; permite duplicados; acceso por índice; buena para lecturas frecuentes.
- **HashSet**: colección de elementos únicos; usa hashing internamente; no mantiene orden.
- **HashMap**: almacena pares clave→valor; claves únicas; operaciones rápidas por hashing; ideal para búsquedas por clave.

Métodos y comandos

ArrayList (java.util.ArrayList)

`ArrayList<String> list = new ArrayList<>();`

`list.add("A");` // Añadir un valor

`list.add(1, "A");` //Añadir un valor en una posición concreta

`list.get(0);` // Obtener un valor por posición

`list.set(0, "B");` //Sustituye el elemento de la posición por un nuevo valor

`list.remove(0);` // Eliminar por posición, también podemos borrar por elemento, por ejemplo "C", buscará la primera C y la borrará.

`list.indexOf("A");` //Devuelve la posición de la primera aparición del elemento

`list.size();` // Tamaño de la lista

`list.contains("A");` // Si contiene o no, booleano

`list.clear();` // Borrar la lista

`list.isEmpty();` //Si está vacía o no, booleano

`list.subList(pos, pos2);` //Sacamos una parte de la lista, desde una posición hasta otra, cabe destacar que si ponemos desde el elemento 0 al 2, será del 0 al 1 ya que el 2 no se incluye.

`list.equals(list2);` //Comprobamos si una lista es exactamente igual a otra en size, valores e índices, booleano.

`list.addAll(list2);` //Une dos listas en una, no borra la que se añade a la principal.

`ArrayList<String> listCopy = new ArrayList<>(list);` //Crea una copia cruda de la lista

`Object copia = list.clone();` //Crea un clon de la copia sin crear un arraylist nuevo

```
System.out.println(list);    //Printeamos la lista en crudo
```

HashSet (java.util.HashSet)

```
HashSet<Integer> set = new HashSet<>();
```

Prácticamente mismos usos que ArrayList

No existe el orden ni el acceso posicional y no se permiten elementos duplicados

Podemos usar un `.add` de un HashSet como booleano, devolverá true si es añadido correctamente, en caso de ya existir el valor devolverá false (No se admiten duplicados)

HashMap (java.util.HashMap)

```
HashMap<String,Integer> map = new HashMap<>();
```

```
map.put(Key, Valor);    // Añade llave y valor, si la llave existe sobrescribe el valor
```

```
map.get(Key);           // Obtiene el valor asociado a la llave
```

```
map.remove(Key);        // Borra llave y valor asociado a la llave
```

```
map.containsKey(Key);   // Devuelve booleano de si el map contiene la llave o no
```

```
map.containsValue(Valor); //Devuelve booleano de si el map contiene el valor o no
```

```
map.getKey();           // Obtenemos la llave del elemento (Usado en recorridos de bucle)
```

```
map.getValue();         // Obtenemos el valor del elemento (Usado en recorridos de bucle)
```

```
map.keySet();           // Devuelve un set con todas las claves
```

```
map.values();           // Devuelve un collection con todos los valores
```

```
map.entrySet();         // Devuelve un set de objetos Map.Entry con pareja Key-Valor
```

Iteración: for-each y Map.entrySet

`Iterator<Type> it = list.iterator();` //Declaración de iterator, en type ponemos el tipo de valor de la lista (String, int, etc.) Este es para ArrayList y HashSet

`Iterator<Map.Entry<TypeKey, TypeValue> it = map.entrySet().iterator();`

//Declaración de iterator en HashMap

`while(it.hasNext()){`

`it.next();` //Avanza de posición y dame el valor, siempre al principio para que avance al 0 ya que empieza en -1 por así decirlo

`String s = it.next();` //Lo asignamos a una variable para guardar el elemento y que nos dé el siguiente

`System.out.println(s);`

`it.remove();` //Borrado seguro, siempre con it, la variable solo da una copia del valor, no la posición real

`Map.Entry<TypeKey, TypeValue> e = it.next();` //Uso de iterator de HashMap

`if (e.getValue() < 0){`

`it.remove();`

`}` //Ejemplo de uso con el iterator de HashMap, y el it.remove sigue siendo igual, borrará tanto llave como valor.

`}`

//Bucle con it.hasNext(), básicamente mientras tenga un siguiente elemento, seguimos. Lo usamos para borrar de forma segura, podemos crear varios iteradores para diferentes lista y recorrerlas juntas

IMPORTANTE: No puedes añadir elementos a la lista o hashset o hashmap mientras la recorres con un Iterator. Solo se usa para borrar de forma segura o comprobar si hay un elemento siguiente.

for-each ()

```
for (String s : list) {  
    System.out.println(s);  
}
```

//Podemos meter condicionales y si s.equals("Hola") entonces lo printeamos o cosas así.

```
for (Integer i : set) {  
    System.out.println(i);  
}
```

```
for (String key: map.keySet()) {  
  
    System.out.println(map.get(key));  
  
}
```

//Ejemplo de for each en los tres tipos, arraylist, hashset y hashmap (el tipo de valor es un ejemplo, cambiará)

Iterar Map con entrySet() (recomendado)

```
for (Map.Entry<TypeKey, TypeValue> e : map.entrySet()) {  
    System.out.println(e.getKey() + " -> " + e.getValue());  
}
```

//TypeKey y value referencia al tipo de atributo de llave y valor, como String o Integer, o en el tema de Orientación a objetos, el valor puede ser una clase.

```
for (TypeKey key : map.keySet()){  
  
    System.out.println(map.get(key));  
  
}
```

```
for (TypeValue valor : map.values()) {  
  
    System.out.println(valor);  
}
```

}

//map.values sirve para acceder únicamente al valor si nos da igual la llave, pero para modificar valores necesitamos la llave así que **map.keySet** o el de **map.entrySet**

Ventaja: evita llamadas repetidas a **map.get(key)** y es más eficiente.

Utilidades de Collections (java.util.Collections)

- **Collections.sort(list);** → Ordenar alfabéticamente o ascendentemente
- **Collections.shuffle(list);** → Mezclar aleatoriamente
- **Collections.reverse(list);** → Revertimos la lista, le damos la vuelta
- **Collections.swap(list, pos1, pos2);** → Intercambiamos valores de posición
- **Collections.max(list)** → Devuelve el elemento máximo de la lista
- **Collections.min(list)** → Devuelve el elemento mínimo de la lista

Buenas prácticas y rendimiento

- Usa **ArrayList** para acceso por índice y listas ordenadas.
- Usa **HashSet** para garantizar unicidad y búsquedas rápidas.
- Usa **HashMap** para asociaciones clave→valor; considera **LinkedHashMap** si necesitas orden de inserción.
- Por favor, limpiad el buffer con un **sc.nextLine()** después de un **sc.nextInt()**.